

1. Del código de alto nivel a la memoria física

Los lenguajes de alto nivel son cada vez más abstractos. En un origen, la programación se hacía directamente en ensamblador, atacando directamente al conjunto de instrucciones del propio procesador. Con la creación y avance de los compiladores, estos se sustituyeron por lenguajes algo más abstractos como COBOL, Pascal, o en última instancia C, incluyendo conceptos como funciones o tipos, e incluso multitarea. Posteriormente, se introdujeron lenguajes más complejos como C++, que añade clases, orientación a objetos, plantillas, y multitud de comodidades en versiones más recientes como programación funcional, pero aún con posibilidades cercanas al hardware. Hoy en día, la mayor parte de los programadores trabajan en lenguajes de aún más alto nivel, como Java o Python, que ni siquiera se ejecutan en su mayor parte directamente sobre hardware, sino sobre máquina virtual, lo cual permite estilos de programación muy flexibles donde el concepto de tipos o funciones se diluye.

Esta evolución la permite, en parte, el *stack* tecnológico que vive debajo: Un procesador complejo, sobre el que se ejecuta un sistema operativo, que a su vez tiene un intérprete instalado al que ofrece interfaces de control, sobre el cual finalmente se ejecuta el lenguaje de nuestra elección.

Sin embargo, sigue habiendo nichos donde tener toda esta pila de herramientas no es posible: uno de ellos son los sistemas empotrados.

Un sistema empotrado, aunque su definición sea en ocasiones difusa, suele tener capacidades muy reducidas, y estar diseñado para funciones muy específicas. Dada la simplicidad de su diseño, los procesadores que incluyen no soportarían (o serían demasiado lentos) un sistema con la complejidad como para ejecutar todos estos lenguajes modernos. Por tanto, aún se siguen utilizando lenguajes más simples y con características más cercanas al hardware. En concreto, el lenguaje rey en este ámbito sigue siendo C. Aún y con todo, C sigue siendo un lenguaje de alto nivel, y conviene entender cómo nos va a permitir trabajar directamente con el *hardware*.

1.1. La estructura del hardware - dispositivos mapeados en memoria

Un procesador, por sí solo, no es más que una máquina de ejecutar instrucciones. Según el conjunto de instrucciones con que diseñemos nuestro procesador, le daremos más o menos capacidades. Aún así, muchas de las capacidades no residen en el propio núcleo en sí, sino en periféricos asociados que liberan de trabajo al *core* ejecutor de tareas.

En las arquitecturas de microcontroladores modernas (como la familia ARM Cortex-M), el procesador utiliza, para comunicarse con dichos periféricos, un esquema denominado *Memory-Mapped I/O* (MMIO). Así, para comunicarse con los periféricos del procesador, no se utilizan instrucciones específicas. En su lugar, todos los periféricos comparten el mismo espacio de direccionamiento de 32 bits (4GB de direcciones) junto con la memoria de programa (Flash) y la memoria de datos (RAM).

- **Memoria Flash** (0x08000000): Almacena el código compilado, constantes, e información persistente. Es memoria no volátil, que se mantiene tras un apagado o reinicio del sistema.
- **Memoria SRAM** (0x20000000): Almacena las variables globales, la pila (*stack*) y el montón (*heap*). Es memoria volátil de acceso rápido, que se reinicia en cada inicio del sistema.

- **Periféricos** (0x40000000 - 0x50000000 y otros): Colección de registros físicos mapeados directamente en direcciones de memoria fijas.

Desde la perspectiva del procesador, escribir en la dirección 0x40020000 no difiere sintácticamente de escribir en una variable almacenada en la RAM. Sin embargo, la circuitería del bus redirige esa operación al controlador del periférico correspondiente (por ejemplo, puerto de entrada/salida de propósito general), provocando un cambio de estado ajeno al procesador, como por ejemplo el encendido de un LED.

1.2. El acceso a memoria - punteros y direcciones

Para interactuar con las direcciones físicas descritas en el modelo MMIO desde el lenguaje C, se emplean punteros. En C, debemos trabajar manualmente con valores numéricos (representando dichas direcciones) para conseguir acceder a las mismas. Un puntero se describe de la siguiente manera:

```
1 #include <stdint.h>
2
3 uint32_t * puntero;
```

NOTA: Es fundamental utilizar tipos de datos con ancho de bits garantizado mediante la librería de la cabecera estándar `<stdint.h>` (como `uint32_t`, `uint16_t`, `uint8_t`), evitando tipos ambiguos como `int` o `long` cuyo tamaño puede variar según arquitectura y compilador.

Un puntero no es más que una dirección de memoria. En este caso, `uint32_t *` representa “dirección de memoria donde hay almacenado un `uint32_t`”. Si queremos cambiar ese valor, guardado en dicha dirección de memoria, haremos:

```
1 *puntero = 20;
```

Y, si queremos cambiar la dirección de memoria a la que apuntamos, haremos:

```
1 puntero = (uint32_t *) 0x40020000U;
```

Así, conociendo la dirección de memoria a través de la cual podemos acceder a un periférico (por ejemplo para encender un LED), podemos escribir en ese preciso lugar de la siguiente manera:

```
1 #include <stdint.h>
2
3 // Macro para poder cambiar fácilmente la dirección
4 #define DIRECCION_OBJETIVO (0x40020000U)
5
6 void encender_led(void) {
7     // 1) creación de nuestro puntero (indicamos el tipo del mismo)
8     uint32_t * p_led = (uint32_t *) DIRECCION_OBJETIVO;
9
10    // 2) Escribimos sobre esa dirección de memoria
11    *p_led = 0x00000001;
12 }
```

En la práctica, se suelen combinar ambas operaciones en una única macro para facilitar la escritura de código:

```

1 #define REGISTRO_LED (*(volatile uint32_t *) 0x40020000U)
2 // Ahora podemos utilizar la macro como si fuera una variable
3 REGISTRO_LED = 0x00000001;

```

Si aún quedan dudas, consulta la Figura 1 para verlo en forma de diagrama.

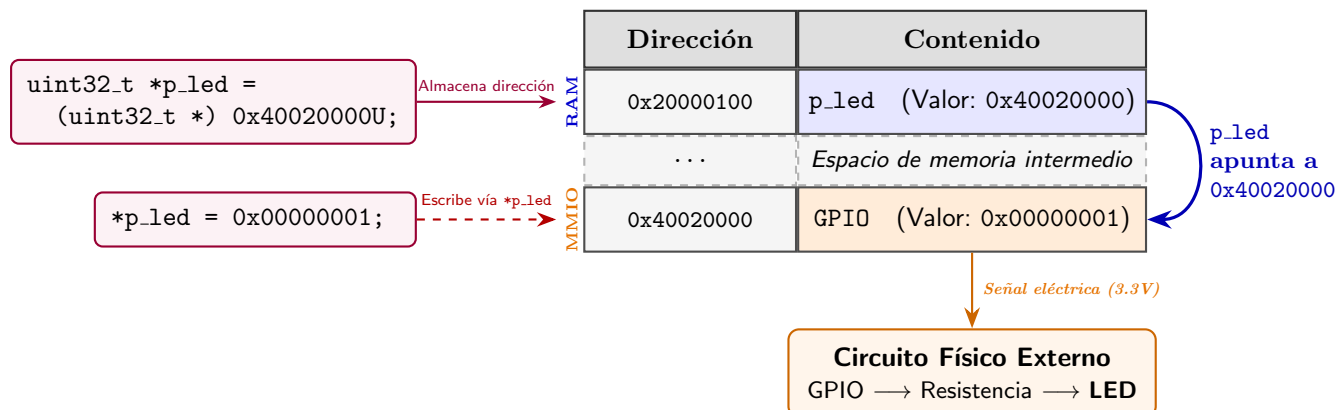


Figura 1: Esquema de las dos direcciones de memoria involucradas con un puntero: la dirección a la que apunta, y el valor apuntado.

NOTA: Si hablamos de punteros, no podemos dejar de lado (aunque no se utilice habitualmente) el operador `&`. Este operador sirve para darnos la *dirección* de una variable. Así, podremos establecer punteros a direcciones concretas donde tenemos almacenados datos:

```

1 uint32_t    a;
2 uint32_t* ptr;
3 ptr = &a; //guardamos dirección de 'a' en el puntero
4 *ptr = 20; //modificamos 'a' directamente

```

1.3. Las operaciones bit a bit - manipulación de registros

Un registro de un periférico de 32 bits suele alojar múltiples configuraciones independientes; cada bit o grupo de bits controla una función específica (por ejemplo, velocidad, dirección de un pin o activación de interrupciones). Por ello, escribir directamente un valor entero (como `REGISTRO_LED = 0x00000001`) suele ser peligroso, ya que sobrescribe y destruye la configuración de los 30 bits restantes.

Para modificar únicamente los bits de interés sin alterar los demás, se utiliza el patrón *Read-Modify-Write* (Lectura-Modificación-Escritura) mediante operadores bit a bit (*bitwise*):

■ Establecer bits a '1':

```

1 //ponemos el bit '0' a '1', resto se mantienen
2 REGISTRO_LED |= 0x00000001;
3 //ponemos todos los bits a '1' salvo el bit '0'
4 //que se mantiene
5 REGISTRO_LED |= 0xFFFFF000;

```

■ Borrar bits a '0':

```

1 //ponemos el bit '0' a '0', resto se mantienen
2 REGISTRO_LED &= ~0x00000001;
3 //borramos todos los bits salvo el bit '0'
4 //que se mantiene
5 REGISTRO_LED &= 0x00000001;

```

■ Alternar estado:

```

1 //cambiamos el bit '0' (de '1' a '0' o de '0' a '1')
2 //resto se mantienen
3 REGISTRO_LED ^= 0x00000001;
4 //cambiamos todos los bits excepto el '0'
5 REGISTRO_LED ^= 0xFFFFF0;

```

■ Leer bits individuales:

```

1 //leemos el bit '0'
2 uint32_t valor = REGISTRO_LED & 0x00000001;
3 //leemos el bit '16' y lo movemos a la derecha
4 uint32_t valor = (REGISTRO_LED & 0x00010000) >> 16;

```

Hay que tener siempre en cuenta que los bits en los que haya que escribir, dependerán del funcionamiento del periférico mapeado en memoria. También, en ocasiones, solo leer el valor desencadena funciones del periférico. Por tanto será muy importante leer su documentación. Imaginemos un registro mapeado en memoria donde hay que modificar los bits 14 y 15 para escribir una configuración:

```

1 // El Pin 7 ocupa los bits [15:14] del registro GPIOA_MODER (2 bits por pin)
2 // Se requiere escribir el valor '01' (salida) en las posiciones 15 y 14.
3
4 // Borrar bits 15 y 14 usando una máscara invertida
5 GPIOA_MODER &= ~(0b11U << (7 * 2));
6 // 0bxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
7 // 0b11111111111111110011111111111111 &
8 // -----
9 // 0bxxxxxxxxxxxxxxxx00xxxxxxxxxxxx
10
11 // Poner el valor '01' en dichos bits
12 GPIOA_MODER |= (0b01U << (7 * 2));
13 // 0bxxxxxxxxxxxxxxxx00xxxxxxxxxxxx
14 // 0b000000000000000010000000000000 |
15 // -----
16 // 0bxxxxxxxxxxxxxxxx01xxxxxxxxxxxx

```

1.4. Definiciones vs variables - el espacio utilizado

En un microcontrolador, los recursos de memoria son escasos (por ejemplo, 512KB de Flash y 1MB de RAM). Es importante conocer qué recursos consume cada elemento en C a fin de ahorrar espacio.

- **Directivas del preprocesador (#define):** No consumen memoria RAM ni Flash por sí mismas. El preprocesador realiza una sustitución textual antes de la compilación.

- **Variables globales no inicializadas (.bss):** Se ubican exclusivamente en la RAM (pues la flash no se puede modificar directamente en tiempo de ejecución). Al arrancar el programa, el código de inicio las inicializa automáticamente a cero.
- **Variables globales inicializadas (.data):** Ocupan espacio en memoria Flash (donde se guarda su valor inicial de fábrica) y en memoria RAM (donde se copian durante el arranque para permitir su modificación).
- **Constantes (const):** Si se declaran con el calificador `const`, el compilador las ubica en la sección de solo lectura de la memoria Flash (`.rodata`), evitando ocupar memoria RAM.
- **Variables locales:** Se crean dinámicamente en la pila (*stack*) ubicada en RAM durante la ejecución de las funciones y se destruyen automáticamente al salir del ámbito.

Es importante conocer que esta distribución no es estrictamente así, ya que cambios en el script de enlazado (o *linker script*) podrán modificarla. Pero en líneas generales podemos asumir que se distribuyen de esta manera.

```

1 #define CONFIG_MASK 0x00FFU           // Sustitución textual, 0 bytes consumidos
2
3 const uint32_t calib = 0xF412AB00;    // Memoria Flash (.rodata), 0 bytes en RAM
4 uint32_t contador_global;             // Memoria RAM (.bss), 4 bytes
5 uint32_t estado_inicial = 0xAA;       // Memoria RAM (.data) + copia en Flash
6
7 void proceso(void) {
8     uint8_t buffer_local[16];         // Stack (en RAM) mientras dure la función
9 }

```